

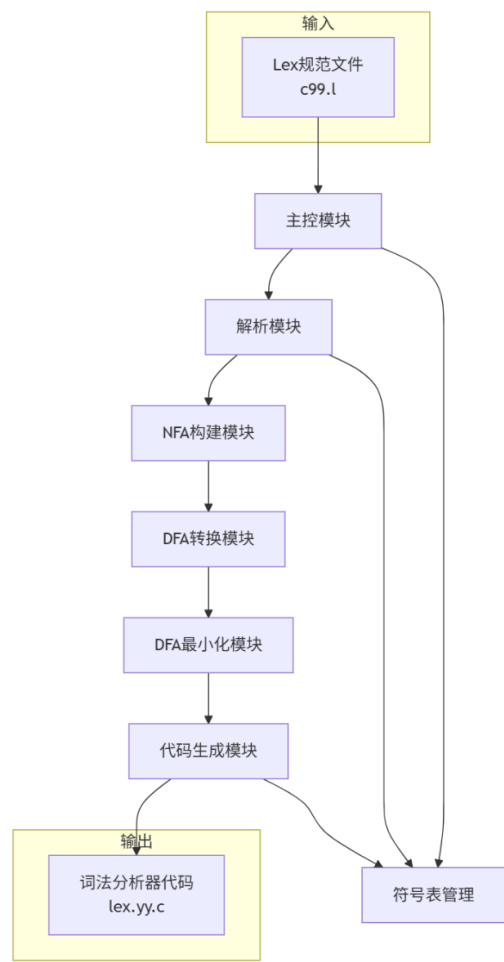
SeuLex词法分析器设计文档

1. 项目概述

SeuLex是一个基于Lex原理的词法分析器生成工具，能够将Lex规范文件（如c99.l）转换为高效的词法分析器代码。本设计文档详细描述了SeuLex的实现方案，包括数据结构设计、整体架构设计和详细实现。

2. 整体架构设计

2.1 模块图



2.2 数据流说明

1. **输入阶段**：读取Lex规范文件（c99.l），包含定义段、规则段和用户代码段。
2. **解析阶段**：解析规范文件，提取正则表达式定义、规则和用户代码。
3. **NFA构建阶段**：使用Thompson构造法将正则表达式转换为NFA。
4. **DFA转换阶段**：使用子集构造法将NFA转换为DFA。
5. **DFA最小化阶段**：使用Hopcroft算法对DFA进行最小化。
6. **代码生成阶段**：根据最小化后的DFA生成词法分析器代码。
7. **符号表管理**：在整个过程中维护符号表，处理保留字和标识符。

3. 数据结构设计

3.1 核心数据结构

3.1.1 规则结构 (Rules)

```
struct Rules {  
    string pattern;           // 原始正则表达式模式  
    string suffix_pattern;    // 后缀形式的正则表达式  
    string action;            // 对应动作代码  
    int priority;             // 优先级（出现顺序）  
};
```

3.1.2 NFA状态结构 (NFAstate)

```
struct NFAstate {  
    int id;                   // 状态唯一标识符  
    unordered_map<char, unordered_set<int>> transitions; // 状态转换映射  
    bool isEnd;               // 是否为接受状态  
    int ruleIndex;            // 对应规则的索引（仅接受状态有效）  
};  
  
struct NFA {  
    int startState;           // 起始状态  
    unordered_map<int, NFAstate> states; // 所有状态  
    map<int, int> endStates;   // 接受状态到规则索引的映射  
};
```

3.1.3 DFA状态结构 (DFAstate)

```
struct DFAstate {  
    int id;                   // 状态唯一标识符  
    unordered_map<char, int> transitions; // 状态转换映射  
    bool isEnd;               // 是否为接受状态  
    int ruleIndex;            // 对应规则的索引（仅接受状态有效）  
    unordered_set<int> nfaStates; // 包含的NFA状态集合  
};  
  
struct DFA {  
    int startState;           // 起始状态  
    vector<DFAstate> states; // 所有状态  
};
```

3.2 辅助数据结构

3.2.1 输入缓冲区

```
class InputBuffer {
private:
    string buffer;           // 输入缓冲区
    int position;            // 当前位置
    int line;                // 当前行号
    int column;              // 当前列号

public:
    InputBuffer(const string& input);
    char getNextChar();      // 获取下一个字符
    void ungetChar();        // 回退一个字符
    int getLine() const;     // 获取当前行号
    int getColumn() const;   // 获取当前列号
};
```

3.2.2 符号表

```
struct SymbolTable {
    map<string, string> definitions; // 正则定义映射

    void addDefinition(const string& name, const string& value);
    string lookup(const string& name);
};
```

4. 详细设计

3.1 Lex输入文件解析 (Lex RE => 常规RE)

3.1.1 Lex规范文件结构

Lex规范文件 (如c99.l) 包含三个部分:

1. **定义段 (Definitions)** : 位于%{和}%之间, 包含C代码和正则定义
2. **规则段 (Rules)** : 位于%%之间, 包含正则表达式和对应的动作
3. **用户代码段 (User Code)** : 位于第二个%%之后, 包含辅助函数

3.1.2 Lex规范解析器数据结构

```
struct LexSpec {
    map<string, string> definitions; // 正则定义映射 {D -> [0-9], L -> [a-zA-Z_], ...}
    vector<Rule> rules;              // 规则列表
    string userCode;                 // 用户代码段
    string headerCode;               // 定义段中的C代码
};

struct Rule {
```

```
string pattern;           // 正则表达式模式 (Lex RE格式)
string normalizedPattern; // 规范化后的正则表达式 (常规RE格式)
string action;            // 对应动作代码
int priority;             // 优先级 (出现顺序)
};
```

3.1.3 Lex规范解析函数

函数签名:

```
LexSpec parseLexSpec(const string& content);
```

输入: Lex规范文件内容 **输出:** 解析后的LexSpec结构 **前置条件:** 文件内容符合Lex规范格式 **后置条件:** 成功提取定义、规则和用户代码

伪代码实现:

```
LexSpec parseLexSpec(const string& content)
{
    LexSpec spec;
    int pos = 0;

    // 跳过空白和注释
    skipWhitespaceAndComments(content, pos);

    // 解析定义段
    if (matchString(content, pos, "%{")
        { pos += 2;
          spec.headerCode = parseUntil(content, pos, "%}");
          pos += 2;
        }

    // 解析正则定义 (如 D [0-9])
    while (pos < content.length() && !matchString(content, pos, "%"))
    { skipWhitespace(content, pos);
      if (content[pos] == '\\n' || content[pos] == '\\r')
      { pos++;
        continue;
      }

      // 解析定义名 (如 D, L, H)
      string name = parseIdentifier(content, pos);
      skipWhitespace(content, pos);

      // 解析定义值 (如 [0-9])
      string value = parseDefinitionValue(content, pos);

      spec.definitions[name] = value;
    }
}
```

```

// 解析规则段 (%% ... %)
if (matchString(content, pos, "%%"))
{ pos += 2;
  skipWhitespace(content, pos);

  int ruleIndex = 0;
  while (pos < content.length() && !matchString(content, pos, "%%"))
  { skipWhitespace(content, pos);

    // 解析正则表达式
    string pattern = parseRegexPattern(content, pos);
    skipWhitespace(content, pos);

    // 解析动作代码 (位于 {} 中)
    string action = parseAction(content, pos);

    Rule rule;
    rule.pattern = pattern;
    rule.action = action;
    rule.priority = ruleIndex++;

    // 将Lex RE转换为常规RE
    rule.normalizedPattern = normalizeLexRegex(pattern, spec.definitions);

    spec.rules.push_back(rule);
  }
}

// 解析用户代码段
if (matchString(content, pos, "%%"))
{ pos += 2;
  spec.userCode = content.substr(pos);
}

return spec;
}

```

3.1.4 Lex RE到常规RE的转换

函数签名:

```

string normalizeLexRegex(const string& lexPattern, const map<string, string>&
definitions);

```

输入:

- lexPattern: Lex格式的正则表达式
- definitions: 正则定义映射

输出: 规范化的常规正则表达式 **前置条件:** Lex RE语法正确 **后置条件:** 返回的RE符合标准正则表达式语法

转换规则:

1. **宏替换**: 将{D}替换为对应的定义[0-9]
2. **字面量引号**: 将"auto"转换为a·u·t·o (连接操作)
3. **字符类展开**: [a-z]保持不变
4. **排除字符类**: [^a-z]转换为补集表示
5. **转义处理**: 处理\n, \t等特殊字符

伪代码实现:

```

string normalizeLexRegex(const string& lexPattern, const map<string, string>&
definitions) {
    string result;
    int pos = 0;

    while (pos < lexPattern.length())
        { char c = lexPattern[pos];

        // 处理宏引用 {D}
        if (c == '{' && pos + 1 < lexPattern.length() && isalpha(lexPattern[pos +
1])) {
            int start = pos + 1;
            int end = start;
            while (end < lexPattern.length() && lexPattern[end] != '}')
                { end++;
            }

            string macroName = lexPattern.substr(start, end - start);
            if (definitions.find(macroName) != definitions.end()) {
                result += "(" + definitions.at(macroName) + ")";
                pos = end + 1;
                continue;
            }
        }

        // 处理引号内的字面量 "auto"
        if (c == '"') {
            pos++;
            string literal;
            while (pos < lexPattern.length() && lexPattern[pos] != '"') {
                if (lexPattern[pos] == '\\' && pos + 1 < lexPattern.length()) {
                    // 处理转义字符
                    literal += lexPattern[pos + 1];
                    pos += 2;
                } else {
                    literal += lexPattern[pos];
                    pos++;
                }
            }
            pos++; // 跳过结束引号

            // 将字面量转换为连接形式

```

```
        for (char lc : literal) {
            if (!result.empty()) result += ".";
            result += lc;
        }
        continue;
    }

    // 处理字符类 [a-z]
    if (c == '[') {
        int start = pos;
        pos++;
        if (lexPattern[pos] == '^')
            { pos++;
            }
        while (pos < lexPattern.length() && lexPattern[pos] != ']')
            { if (lexPattern[pos] == '\\') pos++;
              pos++;
            }
        pos++; // 跳过 ]
        result += lexPattern.substr(start, pos - start);
        continue;
    }

    // 处理括号分组
    if (c == '(') {
        int start = pos;
        int depth = 1;
        pos++;
        while (pos < lexPattern.length() && depth > 0)
            { if (lexPattern[pos] == '(') depth++;
              else if (lexPattern[pos] == ')') depth--;
              else if (lexPattern[pos] == '\\') pos++;
              pos++;
            }

        // 递归处理括号内的内容
        string inner = lexPattern.substr(start + 1, pos - start - 2);
        string normalizedInner = normalizeLexRegex(inner, definitions);
        result += "(" + normalizedInner + ")";
        continue;
    }

    // 处理量词 *, +, ?, {n,m}
    if (c == '*' || c == '+' || c == '?')
    { result += c;
      pos++;
      continue;
    }

    if (c == '{') {
        // 处理 {n} 或 {n,m}
        int start = pos;
        pos++;
        while (pos < lexPattern.length() && lexPattern[pos] != '}') {
```

```

        pos++;
    }
    pos++;
    result += lexPattern.substr(start, pos - start);
    continue;
}

// 处理普通字符
if (!result.empty() && !isOperator(result.back()) && !isOperator(c))
{ result += "."; // 隐式连接
}
result += c;
pos++;
}

return result;
}

```

3.2 常规正则表达式解析（转语法树或后缀表达式）

3.2.1 正则表达式语法树节点

```

enum class RegexNodeType {
    CHAR,           // 单个字符
    CHAR_CLASS,     // 字符类 [a-z]
    CONCAT,         // 连接操作
    UNION,          // 并集操作 (|)
    STAR,           // 闭包 (*)
    PLUS,           // 正闭包 (+)
    QUESTION,       // 可选 (?)
    DOT             // 任意字符 (.)
};

struct RegexNode {
    RegexNodeType type;
    char value;           // 对于CHAR类型
    string charClass;     // 对于CHAR_CLASS类型
    RegexNode* left;      // 左子树
    RegexNode* right;     // 右子树（对于二元操作）

    RegexNode(RegexNodeType t) : type(t), value(0), left(nullptr), right(nullptr) {}
    RegexNode(char c) : type(RegexNodeType::CHAR), value(c), left(nullptr),
right(nullptr) {}
};

```

3.2.2 正则表达式解析器（中缀转后缀）

函数签名：


```
string infixToPostfix(const string& infix);
```

输入：中缀形式的正则表达式（已规范化） **输出：**后缀形式的正则表达式 **前置条件：**正则表达式语法正确 **后置条件：**返回的后缀表达式保持语义等价

伪代码实现：

```
string infixToPostfix(const string& infix)
{
    string postfix;
    stack<char> opStack;

    // 定义运算符优先级
    auto precedence = [](char op) -> int
    {
        switch (op) {
            case '*': case '+': case '?': return 3; // 最高优先级
            case '.': return 2; // 连接操作
            case '|': return 1; // 并集操作
            default: return 0;
        }
    };

    int i = 0;
    while (i < infix.length())
    {
        char c = infix[i];

        // 处理字符类 [a-z]
        if (c == '[') {
            int start = i;
            i++;
            if (infix[i] == '^') i++;
            while (i < infix.length() && infix[i] != ']')
            {
                if (infix[i] == '\\') i++;
                i++;
            }
            i++; // 跳过 ]
            postfix += infix.substr(start, i - start);
            continue;
        }

        // 处理括号
        if (c == '(') {
            opStack.push(c);
            i++;
            continue;
        }

        if (c == ')') {
            while (!opStack.empty() && opStack.top() != '(')
            {
                postfix += opStack.top();
                opStack.pop();
            }
        }
    }
}
```

```
    }
    opStack.pop(); // 弹出 '('
    i++;
    continue;
}

// 处理量词
if (c == '*' || c == '+' || c == '?') {
    while (!opStack.empty() && precedence(opStack.top()) >= precedence(c))
    {
        postfix += opStack.top();
        opStack.pop();
    }
    opStack.push(c);
    i++;
    continue;
}

// 处理连接操作符 '.'
if (c == '.') {
    while (!opStack.empty() && precedence(opStack.top()) >= precedence(c))
    {
        postfix += opStack.top();
        opStack.pop();
    }
    opStack.push(c);
    i++;
    continue;
}

// 处理并集操作符 '|'
if (c == '|') {
    while (!opStack.empty() && precedence(opStack.top()) >= precedence(c))
    {
        postfix += opStack.top();
        opStack.pop();
    }
    opStack.push(c);
    i++;
    continue;
}

// 处理普通字符
postfix += c;
i++;
}

// 弹出剩余的运算符
while (!opStack.empty()) {
    postfix += opStack.top();
    opStack.pop();
}
```

```
    return postfix;
}
```

3.2.3 从后缀表达式构建语法树

函数签名:

```
RegexNode* buildSyntaxTree(const string& postfix);
```

输入: 后缀形式的正则表达式 **输出:** 语法树的根节点 **前置条件:** 后缀表达式语法正确 **后置条件:** 返回的语法树能正确表示正则表达式的结构

伪代码实现:

```
RegexNode* buildSyntaxTree(const string& postfix)
{
    stack<RegexNode*> nodeStack;
    int i = 0;

    while (i < postfix.length())
    {
        char c = postfix[i];

        // 处理字符类 [a-z]
        if (c == '[') {
            int start = i;
            i++;
            if (postfix[i] == '^') i++;
            while (i < postfix.length() && postfix[i] != ']')
                { if (postfix[i] == '\\') i++; i++; }
            i++; // 跳过 ]

            RegexNode* node = new RegexNode(RegexNodeType::CHAR_CLASS);
            node->charClass = postfix.substr(start, i - start);
            nodeStack.push(node);
            continue;
        }

        // 处理一元操作符
        if (c == '*' || c == '+' || c == '?')
        {
            RegexNodeType type;
            switch (c) {
                case '*': type = RegexNodeType::STAR; break;
                case '+': type = RegexNodeType::PLUS; break;
                case '?': type = RegexNodeType::QUESTION; break;
            }

            RegexNode* operand = nodeStack.top();
            nodeStack.pop();
```

```

        RegexNode* node = new RegexNode(type);
        node->left = operand;
        nodeStack.push(node);
        i++;
        continue;
    }

    // 处理二元操作符
    if (c == '.' || c == '|') {
        RegexNodeType type = (c == '.') ? RegexNodeType::CONCAT :
RegexNodeType::UNION;

        RegexNode* right = nodeStack.top();
        nodeStack.pop();
        RegexNode* left = nodeStack.top();
        nodeStack.pop();

        RegexNode* node = new RegexNode(type);
        node->left = left;
        node->right = right;
        nodeStack.push(node);
        i++;
        continue;
    }

    // 处理普通字符
    RegexNode* node = new RegexNode(c);
    nodeStack.push(node);
    i++;
}

return nodeStack.top();
}

```

3.2.4 解析c99.l的示例

以c99.l中的规则为例，展示解析过程：

示例1：标识符规则

```
{L}({L}|{D})*
```

解析步骤：

1. **Lex RE展开**: $[a-zA-Z_][a-zA-Z_][0-9]^*$
2. **添加隐式连接**: $[a-zA-Z_] \cdot ([a-zA-Z_][0-9]^*)$
3. **中缀转后缀**: $[a-zA-Z_][a-zA-Z_][0-9]^* \cdot$

示例2：十六进制整数

```
0[xX]{H}+{IS}?
```

解析步骤:

1. **Lex RE展开**: $0[xX][a-zA-Z0-9]+((u|U)|(u|U)?(1|L|11|LL)|(1|L|11|LL)(u|U))?$
2. **规范化**: $0 \cdot [xX] \cdot [a-zA-Z0-9]+ \cdot ((u|U)|(u|U)?(1|L|11|LL)|(1|L|11|LL)(u|U))?$
3. **中缀转后缀**: $0[xX][a-zA-Z0-9]+(u|U)(u|U)?(1|L|11|LL)|(1|L|11|LL)(u|U)||? \cdot$

示例3: 字符串字面量

```
L?"(\\.|[^\\"n])*\"
```

解析步骤:

1. **Lex RE展开**: $L?(\"(\\.|[^\\"n])*\")$
2. **规范化**: $L? \cdot \" \cdot (\\.|[^\\"n]) * \cdot \"$
3. **中缀转后缀**: $L? \" \\.|[^\\"n] * \" \cdot$

3.3 正则表达式解析与NFA构建

3.1.1 正则表达式解析函数

函数签名:

```
NFAState* parseRegex(const string& regex);
```

输入: 正则表达式字符串 **输出**: NFA的起始状态 **前置条件**: 正则表达式语法正确 **后置条件**: 返回的NFA能正确识别该正则表达式

伪代码实现:

```
NFAState* parseRegex(const string& regex)
{
    stack<NFAState*> stateStack;
    stack<char> opStack;

    for (char c : regex)
    {
        if (isOperand(c))
        {
            // 创建单个字符的NFA
            NFAState* start = new NFAState();
            NFAState* end = new NFAState();
            start->transitions[c].push_back(end);
            stateStack.push(start);
            stateStack.push(end);
        } else if (c == '(')
        {
            opStack.push(c);
        }
    }
}
```

```

        } else if (c == ')') {
            while (!opStack.empty() && opStack.top() != '(')
                { applyOperator(opStack, stateStack);
            }
            opStack.pop(); // 弹出 '('
        } else {
            while (!opStack.empty() && precedence(opStack.top()) >= precedence(c))
            {
                applyOperator(opStack, stateStack);
            }
            opStack.push(c);
        }
    }

    while (!opStack.empty()) {
        applyOperator(opStack, stateStack);
    }

    return stateStack.top();
}

```

3.1.2 Thompson构造法实现

函数签名:

```
NFAState* thompsonConstruct(const string& regex);
```

输入: 正则表达式字符串 **输出:** NFA的起始状态 **前置条件:** 正则表达式语法正确 **后置条件:** 返回的NFA能正确识别该正则表达式

伪代码实现:

```

NFAState* thompsonConstruct(const string& regex) {
    // 处理连接操作符 (隐式)
    string processedRegex = insertConcatenationOperators(regex);

    // 转换为后缀表达式
    string postfix = infixToPostfix(processedRegex);

    stack<pair<NFAState*, NFAState*>> nfaStack;

    for (char c : postfix)
    { if (isOperand(c)) {
        // 单个字符
        NFAState* start = new NFAState();
        NFAState* end = new NFAState();
        start->transitions[c].push_back(end);
        nfaStack.push({start, end});
    } else if (c == '*') {

```

```
// 闭包操作
auto [start, end] = nfaStack.top();
nfaStack.pop();

NFAState* newStart = new NFAState();
NFAState* newEnd = new NFAState();

// ε转换
newStart->transitions['ε'].push_back(start);
newStart->transitions['ε'].push_back(newEnd);
end->transitions['ε'].push_back(start);
end->transitions['ε'].push_back(newEnd);

nfaStack.push({newStart, newEnd});
} else if (c == '+') {
    // 正闭包操作
    auto [start, end] = nfaStack.top();
    nfaStack.pop();

    NFAState* newStart = new NFAState();
    NFAState* newEnd = new NFAState();

    // ε转换
    newStart->transitions['ε'].push_back(start);
    end->transitions['ε'].push_back(start);
    end->transitions['ε'].push_back(newEnd);

    nfaStack.push({newStart, newEnd});
} else if (c == '?') {
    // 可选操作
    auto [start, end] = nfaStack.top();
    nfaStack.pop();

    NFAState* newStart = new NFAState();
    NFAState* newEnd = new NFAState();

    // ε转换
    newStart->transitions['ε'].push_back(start);
    newStart->transitions['ε'].push_back(newEnd);
    end->transitions['ε'].push_back(newEnd);

    nfaStack.push({newStart, newEnd});
} else if (c == '|') {
    // 选择操作
    auto [start2, end2] = nfaStack.top();
    nfaStack.pop();
    auto [start1, end1] = nfaStack.top();
    nfaStack.pop();

    NFAState* newStart = new NFAState();
    NFAState* newEnd = new NFAState();

    // ε转换
    newStart->transitions['ε'].push_back(start1);
```

```

        newStart->transitions['ε'].push_back(start2);
        end1->transitions['ε'].push_back(newEnd);
        end2->transitions['ε'].push_back(newEnd);

        nfaStack.push({newStart, newEnd});
    } else if (c == '.') {
        // 连接操作
        auto [start2, end2] = nfaStack.top();
        nfaStack.pop();
        auto [start1, end1] = nfaStack.top();
        nfaStack.pop();

        // 连接两个NFA
        end1->transitions['ε'].push_back(start2);

        nfaStack.push({start1, end2});
    }
}

auto [start, end] = nfaStack.top();
end->isAccept = true;
return start;
}

```

3.4 NFA到DFA的转换

3.2.1 子集构造法实现

函数签名:

```
DFAState* nfaToDfa(NFAState* nfaStart);
```

输入: NFA的起始状态 **输出:** DFA的起始状态 **前置条件:** NFA结构完整 **后置条件:** 返回的DFA与原NFA等价

伪代码实现:

```

DFAState* nfaToDfa(NFAState* nfaStart)
{
    map<set<int>, DFAState*> stateMap;
    queue<DFAState*> stateQueue;

    // 计算初始ε-闭包
    set<int> initialStates = epsilonClosure({nfaStart});
    DFAState* initialDfaState = new DFAState();
    initialDfaState->nfaStates = initialStates;
    initialDfaState->isAccept = hasAcceptState(initialStates);
    initialDfaState->ruleIndex = getRuleIndex(initialStates);

    stateMap[initialStates] = initialDfaState;
    stateQueue.push(initialDfaState);
}

```



```

while (!stateQueue.empty()) {
    DFAState* current = stateQueue.front();
    stateQueue.pop();

    // 获取所有可能的输入字符
    set<char> inputChars = getInputChars(current->nfaStates);

    for (char c : inputChars)
        { if (c == 'ε')
          continue;

          // 计算移动后的状态集
          set<int> moveStates = move(current->nfaStates, c);
          set<int> closureStates = epsilonClosure(moveStates);

          // 检查是否已存在该状态集
          if (stateMap.find(closureStates) == stateMap.end())
              { DFAState* newState = new DFAState();
                newState->nfaStates = closureStates;
                newState->isAccept = hasAcceptState(closureStates);
                newState->ruleIndex = getRuleIndex(closureStates);

                stateMap[closureStates] = newState;
                stateQueue.push(newState);
              }

          // 添加转换
          current->transitions[c] = stateMap[closureStates];
        }
    }

    return initialDfaState;
}

```

3.5 DFA最小化

3.3.1 Hopcroft算法实现

函数签名:

```
DFAState* minimizeDfa(DFAState* dfaStart);
```

输入: DFA的起始状态 **输出:** 最小化后的DFA起始状态 **前置条件:** DFA结构完整 **后置条件:** 返回的DFA是最小化的, 与原DFA等价

伪代码实现:

```

DFAState* minimizeDfa(DFAState* dfaStart) {
    // 收集所有DFA状态

```

```
set<DFAState*> allStates = collectStates(dfaStart);

// 初始化划分
vector<set<DFAState*>> partitions;
set<DFAState*> acceptStates, nonAcceptStates;

for (DFAState* state : allStates)
{ if (state->isAccept) {
    acceptStates.insert(state);
  } else {
    nonAcceptStates.insert(state);
  }
}

if (!acceptStates.empty()) partitions.push_back(acceptStates);
if (!nonAcceptStates.empty()) partitions.push_back(nonAcceptStates);

// 迭代划分
bool changed;
do {
    changed = false;
    vector<set<DFAState*>> newPartitions;

    for (const set<DFAState*>& partition : partitions)
    { if (partition.size() <= 1) {
        newPartitions.push_back(partition);
        continue;
      }

      // 按转换进行划分
      map<map<char, int>, set<DFAState*>> groups;

      for (DFAState* state : partition)
      { map<char, int> signature;
        for (const auto& [c, next] : state->transitions) {
            // 找到next所在的分区索引
            int partitionIndex = findPartitionIndex(partitions, next);
            signature[c] = partitionIndex;
        }
        groups[signature].insert(state);
      }

      for (const auto& [_, group] : groups)
      { newPartitions.push_back(group);
      }

      if (groups.size() > 1) changed = true;
    }

    partitions = newPartitions;
} while (changed);

// 构建最小化DFA
```

```
        return buildMinimizedDfa(partitions, dfaStart);  
    }
```

3.6 代码生成

3.4.1 词法分析器代码生成函数

函数签名:

```
void generateCode(DFAState* dfaStart, const vector<Rule>& rules, const string&  
userCode, const string& outputFile);
```

输入:

- dfaStart: 最小化后的DFA起始状态
- rules: 规则集合
- userCode: 用户代码段
- outputFile: 输出文件名

输出: 无 (生成文件) **前置条件:** DFA结构完整, 规则和用户代码有效 **后置条件:** 生成的词法分析器代码能正确识别输入的词法单元

伪代码实现:

```
void generateCode(DFAState* dfaStart, const vector<Rule>& rules, const string&  
userCode, const string& outputFile) {  
    ofstream out(outputFile);  
  
    // 写入头部  
    out << "#include <stdio.h>\n";  
    out << "#include <string.h>\n";  
    out << "#include <stdlib.h>\n";  
    out << "\n";  
  
    // 写入用户代码  
    out << userCode << "\n";  
  
    // 写入状态定义  
    out << "// 状态定义\n";  
    map<DFAState*, int> stateMap;  
    int stateId = 0;  
    assignStateIds(dfaStart, stateMap, stateId);  
  
    // 写入状态转换表  
    out << "// 状态转换表\n";  
    out << "static int transition[][256] = {\n";  
    for (const auto& [state, id] : stateMap) {  
        out << "    {";  
        for (int c = 0; c < 256; c++) {
```

```

        if (state->transitions.find(c) != state->transitions.end())
            { out << stateMap[state->transitions.at(c)];
            } else {
                out << "-1";
            }
        if (c < 255) out << ", ";
    }
    out << "},\n";
}
out << "};\n\n";

// 写入接受状态表
out << "/* 接受状态表\n";
out << "static int acceptState[] = {";
for (const auto& [state, id] : stateMap) {
    out << (state->isAccept ? state->ruleIndex : -1);
    if (id < stateId - 1) out << ", ";
}
out << "};\n\n";

// 写入词法分析器主函数
out << "int yylex() {\n";
out << "    static char buffer[1024];\n";
out << "    int state = 0;\n";
out << "    int pos = 0;\n";
out << "    char c;\n";
out << "\n";
out << "    while ((c = getchar()) != EOF) {\n";
out << "        buffer[pos++] = c;\n";
out << "        int nextState = transition[state][(unsigned char)c];\n";
out << "        if (nextState == -1) {\n";
out << "            // 回退字符\n";
out << "            ungetc(c, stdin);\n";
out << "            buffer[pos-1] = '\\0';\n";
out << "            int ruleIndex = acceptState[state];\n";
out << "            if (ruleIndex != -1) {\n";
out << "                // 执行对应动作\n";
out << "                switch (ruleIndex) {\n";
for (int i = 0; i < rules.size(); i++) {
    out << "                    case " << i << ":\n";
    out << "                        " << rules[i].action << "\n";
    out << "                    break;\n";
}
out << "                }\n";
out << "            } else {\n";
out << "                // 处理未识别字符\n";
out << "                printf(\"Unrecognized character: %c\\n\",
buffer[0]);\n";
out << "            }\n";
out << "            pos = 0;\n";
out << "            state = 0;\n";
out << "        } else {\n";
out << "            state = nextState;\n";
out << "        }\n";

```

```

    out << "    }\n";
    out << "    \n";
    out << "    // 处理文件结束\n";
    out << "    if (pos > 0) {\n";
    out << "        buffer[pos] = '\\0';\n";
    out << "        int ruleIndex = acceptState[state];\n";
    out << "        if (ruleIndex != -1) {\n";
    out << "            switch (ruleIndex) {\n";
    for (int i = 0; i < rules.size(); i++) {
        out << "                case " << i << ":\n";
        out << "                " << rules[i].action << "\n";
        out << "                break;\n";
    }
    out << "            }\n";
    out << "        }\n";
    out << "    }\n";
    out << "    \n";
    out << "    return 0;\n";
    out << "}\n";

    out.close();
}

```

3.7 主函数流程

函数签名:

```
int main(int argc, char* argv[]);
```

输入: 命令行参数 (输入文件和输出文件) **输出:** 执行状态 **前置条件:** 输入文件存在且格式正确 **后置条件:** 生成词法分析器代码

伪代码实现:

```

int main(int argc, char* argv[])
{ if (argc != 3) {
    fprintf(stderr, "Usage: %s input.l output.c\n", argv[0]);
    return 1;
}

// 读取输入文件
string input = readFile(argv[1]);

// 解析Lex规范
LexSpec spec = parseLexSpec(input);

// 构建NFA
vector<NFAState*> nfaStarts;
for (const Rule& rule : spec.rules) {

```

```
        NFASState* start = thompsonConstruct(rule.pattern);
        nfaStarts.push_back(start);
    }

    // 合并NFA
    NFASState* mergedNfa = mergeNFAs(nfaStarts, spec.rules);

    // 转换为DFA
    DFASState* dfaStart = nfaToDfa(mergedNfa);

    // 最小化DFA
    DFASState* minimizedDfa = minimizeDfa(dfaStart);

    // 生成代码
    generateCode(minimizedDfa, spec.rules, spec.userCode, argv[2]);

    printf("Lexical analyzer generated successfully: %s\n", argv[2]);
    return 0;
}
```

4. 结论

本设计文档详细描述了Lex词法分析器的实现方案，包括数据结构设计、整体架构设计和详细设计。通过Thompson构造法、子集构造法和Hopcroft算法，实现了从正则表达式到最小化DFA的转换，并生成高效的词法分析器代码。该设计支持所有要求的功能，包括闭包、正闭包、可选、字符集、排除字符集和任意字符。

通过本设计，可以生成能够正确识别C语言词法单元的词法分析器，为编译器前端提供可靠的词法分析功能。